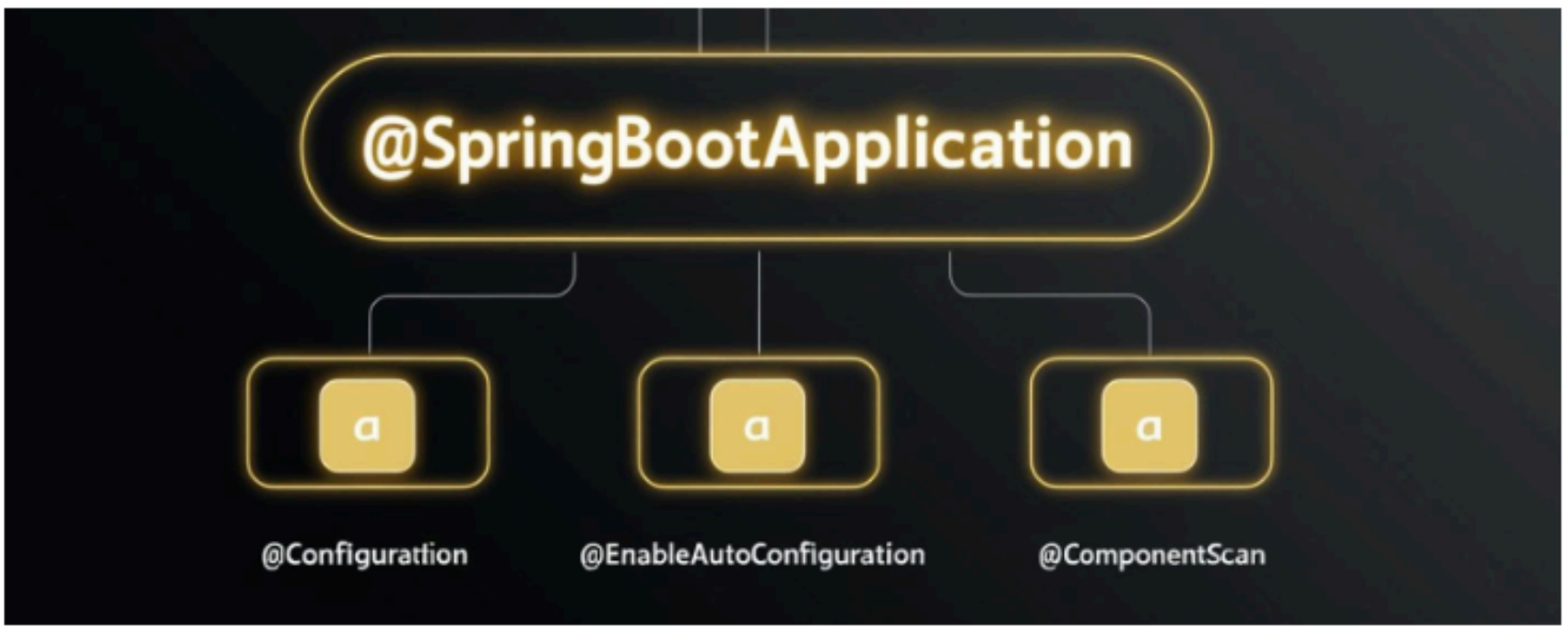
@SpringBootApplication

@SpringBootApplication



- It is a class-level annotation
- It is used to mark a configuration class that declares one or more @Bean methods
- And also triggers auto-configuration and component scanning.

@SpringBootApplication is a Hybrid Annotation

Because it is a combination 3 annotations

- @Configuration
- @ComponentScan
- @EnableAutoConfiguration

@Configuration	▪ This annotation marks our class as a Configuration class for Java-based configuration. ▪ It is used to indicates our class has @Bean definition methods. ▪ @Configuration annotation tells Spring container that there are one or more beans that needs to be dealt with on runtime.
@ComponentScan	▪ This annotation enables component-scanning ▪ It is used to specify the packages that we want to be scanned. ▪ Spring container will automatically scan a package for beans if component scanning is enabled.
@EnableAutoConfiguration	▪ This annotation enables the auto-configuration feature of Spring Boot ▪ Based on the jar files and files in the class path this annotation automatically configure a lot of stuff for you. ▪ Example: It will automatically create, and registers beans based on both the included jar files in the classpath.

@Component

@Component

- It is a class-level annotation.
- When we use @Component on a class, Spring automatically detects it during component scanning.
- The Spring container creates a bean for the annotated class, making it available for dependency injection.

Sample Code

```java
import org.springframework.stereotype.Component;

@Component
public class MyComponent
{
    public void showMessage()
    {
        System.out.println("Hello from @Component Bean!");
    }
}
```

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class MyService
{
    @Autowired
    private MyComponent myComponent;

    public void execute()
    {
        myComponent.showMessage();
    }
}
```

How it works

- Spring automatically detects @Component during component scanning.
- A Spring bean is created for MyComponent.
- We can inject this bean anywhere using @Autowired.

@Controller

@Controller Annotation

- @Controller is a specialized form of @Component annotation.
- It is used to define Spring MVC controllers in the controller layer.
- Marks a class as a Spring-managed bean that handles web requests.

Sample Code

```java
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseBody;

@Controller
@RequestMapping("/home")
public class HomeController
{
    @GetMapping
    @ResponseBody
    public String welcome()
    {
        return "Welcome to Spring MVC!";
    }
}
```

📌 Project Structure for Controller Layer

```
com.example.myapp
├── controller              // Controller Layer – Handles HTTP requests
│   ├── UserController.java   (@Controller)
│   └── OrderController.java  (@Controller)
│
├── service                 // Calls business logic from controller
│   └── UserService.java      (@Service)
│
├── repository              // Data access layer
│   └── UserRepository.java   (@Repository)
│
├── model                   // Defines data models (DTOs, Entities)
│   └── User.java
│
└── Application.java        // Main Spring Boot Application (@SpringBootApplication)
```

@Service

@Service Annotation

- The @Service annotation is a type of @Component annotation.
- It is used to create Spring beans at the Service layer.
- @Service annotation indicates service classes in service layer

Sample Code

```
import org.springframework.stereotype.Service;

@Service
public class UserService
{
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository)
    {
        this.userRepository = userRepository;
    }

    public String getUserById(String id)
    {
        return userRepository.findUserById(id);
    }
}
```

📌 Project Structure for Service Layer

```
com.example.myapp
├── controller           // Calls the service layer
│   └── UserController.java   (@Controller)
│
├── service              // Business logic layer
│   ├── UserService.java      (@Service)
│   └── OrderService.java     (@Service)
│
├── repository           // Data access layer
│   ├── UserRepository.java   (@Repository)
│   └── OrderRepository.java  (@Repository)
│
├── model                // Defines data models (DTOs, Entities)
│   └── User.java
│
└── Application.java     // Main Spring Boot Application (@SpringBootApplication)
```

@Repository

@Repository Annotation

- The @Repository annotation is a type of @Component annotation.
- It is used to create Spring beans for the repositories at the DAO layer.
- @ Repository annotation indicates Repository classes in DAO layer

Sample Code

```java
import org.springframework.stereotype.Repository;
import java.util.HashMap;
import java.util.Map;

@Repository
public class UserRepository
{
    private final Map<String, String> userDatabase = new HashMap<>();

    public UserRepository()
    {
        userDatabase.put("1", "John Tech Community");
        userDatabase.put("2", "PKC");
    }

    public String findUserById(String id)
    {
        return userDatabase.getOrDefault(id, "User Not Found");
    }
}
```

📌 Project Structure for Repository Layer

```
com.example.myapp
├── controller          // Calls the service layer
│   └── UserController.java   (@Controller)
│
├── service             // Calls repository layer
│   └── UserService.java      (@Service)
│
├── repository          // Data access layer
│   ├── UserRepository.java   (@Repository)
│   └── OrderRepository.java  (@Repository)
│
├── model               // Defines data models (DTOs, Entities)
│   ├── User.java
│   └── Order.java
│
└── Application.java    // Main Spring Boot Application (@SpringBootApplication)
```

@Primary

@Primary

- The @Primary annotation is used to resolve the conflict during auto wiring process
- If we have two or more same type of beans configured during Auto wiring process, we can resolve conflict using @ Primary Annotation by giving the high priority to one of the beans
- It is used to give priority to a particular bean, so that it is used as the primary bean for during auto wiring process.

Example 1:

```java
@Service
@Primary
public class PrimaryService implements MyService
{
    @Override
    public void doSomething()
    {
        System.out.println("Doing something from PrimaryService");
    }
}

@Service
public class SecondaryService implements MyService
{
    @Override
    public void doSomething()
    {
        System.out.println("Doing something from SecondaryService");
    }
}

public interface MyService
{
    void doSomething();
}
```

- In this example, both Primary Service and SecondaryService are beans of type MyService.
- The PrimaryService is annotated with the @Primary annotation, which means that it should be given priority when auto wiring MyService.
- This means that, if a class needs an instance of MyService, the PrimaryService will be used as the primary bean.
- If the PrimaryService is not available, the SecondaryService will be used as a fallback.

@Qualifier

@Qualifier

- The @Qualifier annotation is used to resolve the conflict during auto wiring process
- If we have two or more same type of beans configured during Auto wiring process, we can resolve conflict using @Qualifier Annotation
- If we do not handle this case will get **NoUniqueBeanDefinitionException** at run time

Example Code:

```java
interface MessageService
{
    void sendMessage(String message);
}

@Service
@Qualifier("email")
public class EmailMessageService implements MessageService
{
    @Override
    public void sendMessage(String message)
    {
        System.out.println("Sending message through email: " + message);
    }
}

@Service
@Qualifier("sms")
public class SmsMessageService implements MessageService
{
    @Override
    public void sendMessage(String message)
    {
        System.out.println("Sending message through SMS: " + message);
    }
}
```

```java
@Service
public class MessageSender
{
    private final MessageService messageService;

    @Autowired
    public MessageSender(@Qualifier("email") MessageService messageService)
    {
        this.messageService = messageService;
    }

    public void sendMessage(String message)
    {
        messageService.sendMessage(message);
    }
}
```

PURPOSE OF THIS DOCUMENT

- **Use this PDF as a quick last-minute revision guide only.**
- **For more details, please perform R&D and gain a deeper understanding on each topic.**
- **Once the interview series is complete, we will cover this topic in detail.**